MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report
of
Database Systems Lab (CSS 2212)

ONLINE SHOPPING CART MANAGEMENT SYSTEM

SUBMITTED
BY

AKSHITA KAMBHAMPATI, REG.NO: 240953049, CCE-A, 07
NEHA DAS, REG.NO: 240953396, CCE-A, 30

School of Computer Engineering
Manipal Institute of Technology, Manipal.
April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

Manipal
06/04/2026

CERTIFICATE

This is to certify that the project titled **Online Shopping Cart Management System** is a record of the Bonafide work done by **Akshita Kambhampati (240953049) and Neha Das (240953396)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in COMPUTER AND COMMUNICATION ENGINEERING of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

1. **Dr. Anup Bhat B, Assistant Professor, SCE.**
2. **Dr. Sindhura D N, SCE**
3. **Dr. Tyson Baptist D Cunha, SCE**

ABSTRACT

This online shopping portal is a complete online shopping program that allows users to efficiently perform many items of online shopping activities. Users will be able to see all the products they want in their shopping carts before eventually purchasing.

An Administrator will manage product information and product stock for each item in the Shopping Cart System. This gives the system the ability to sell products to customers when there is available stock and will keep accurate records of all purchases by users.

The Shopping Cart System contains a database that allows customers, products, shopping cart items, and order information to be maintained in a manner that allows easy retrieval of the records, at the same time maintaining a consistent record across all users and transactions.

This project illustrates how a DBMS can be used for managing a real-world application with multiple users that perform more than one transaction.

ACM Taxonomy:

- Database Management
- Data Integrity
- Transaction Processing

Sustainable Development Goal (SDG):

- Industry, Innovation and Infrastructure

TABLE OF CONTENTS

LIST OF TABLES

ENTITIES:

- USER
- PRODUCT
- SHOPPING_CART
- CART_ITEM
- ORDER
- ORDER_ITEM
- CATEGORY

TABLES:

1. USER (user_id PK, name, email, phone, address, password)
2. PRODUCT (product_id PK, name, description, price, stock_qty, category)
3. CART (cart_id PK, customer_id FK, created_at, status)
4. CART_ITEM (cart_item_id PK, cart_id FK, product_id FK, quantity)
5. ORDER (order_id PK, customer_id FK, order_date, total_amount, status, payment_method)
6. ORDER_ITEM (order_item_id PK, order_id FK, product_id FK, quantity, unit_price)
7. CATEGORY(category_id, category_name)

LIST OF FIGURES

Abbreviations

DBMS – Database Management System
SQL – Structured Query Language
DDL – Data Definition Language
DML – Data Manipulation Language
ER – Entity Relationship
PK – Primary Key
FK – Foreign Key
UI – User Interface
JDBC – Java Database Connectivity

PL/SQL – Procedural Language / SQL
GUI – Graphical User Interface

CHAPTER 1

INTRODUCTION

1.1 Introduction:

Shop Online is a necessity now and this Online Shopping Cart Management System project's goal is to develop an efficient way for customers to browse, manage their shopping carts, and place orders on the web.

There are two types of users in this system, customers and administrators. Customers will be able to browse through available products, add items to their shopping carts and make purchases, while Administrators will maintain product information and stock levels.

This system will use a Database Management System (DBMS) that stores and manages all data related to users, products, shopping carts, and order history. In this way we can ensure that all data is secure and follows proper protocols; therefore, providing a reliable and efficient way of storing and accessing all information.

1.2 **Need for System**:

As more and more people shop online, there is a need for efficient systems to handle large amounts of user and product data. Errors like wrong stock updates, duplicate records, and failed transactions can happen when systems are not well-organised or are done by hand.

A good database system makes sure that:
Quick access to data
Managing stock correctly

Transactions that are safe

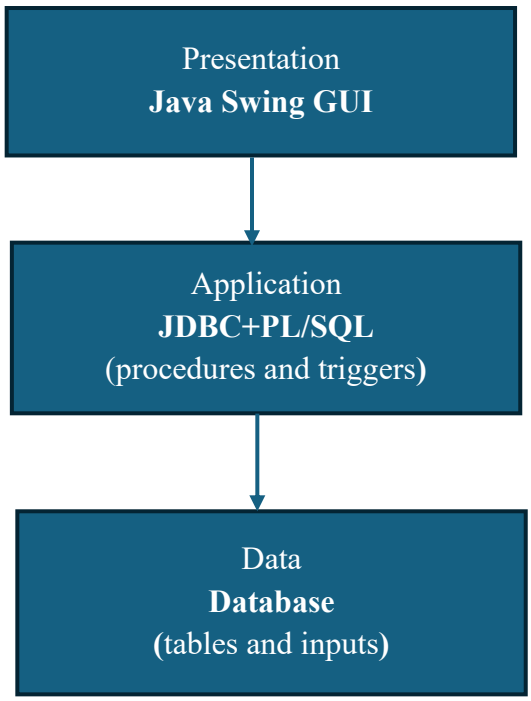
A better experience for users

1.3 Scope:

The system focuses on managing:

- User accounts
- Product inventory
- Shopping cart operations
- Order processing

Figure 1.1: Three Tier Architecture



CHAPTER 2
PROBLEM STATEMENT & OBJECTIVES

3.1 **Problem Statement**:

When you shop the old-fashioned way, keeping track of product availability, user data, and transactions by hand can cause mistakes and waste time. We need a system that can automate shopping, keep data safe, and make the experience for users as smooth as possible.

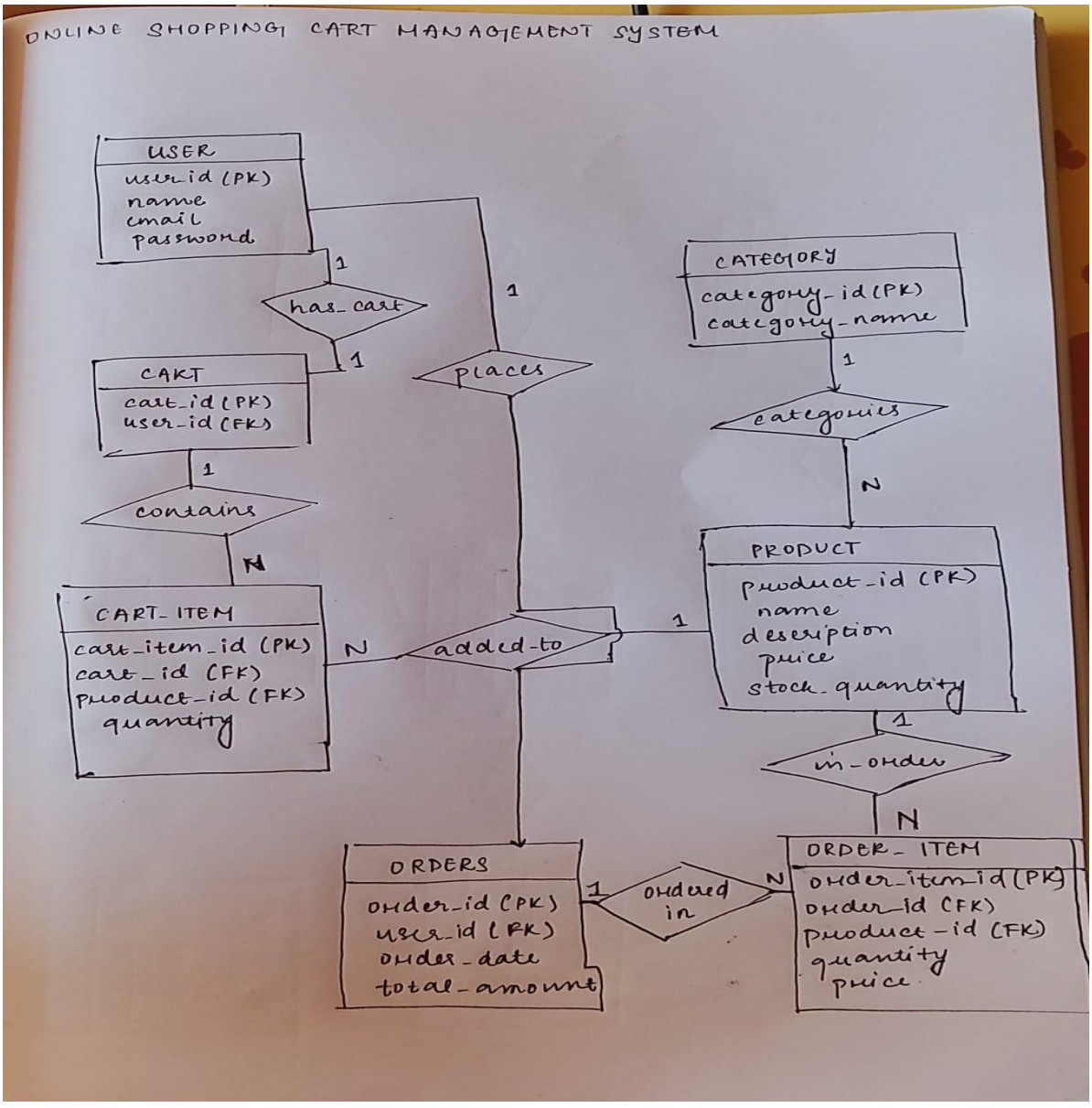
The Online Shopping Cart Management System solves these problems by giving you a structured database to use to keep track of products, users, and transactions.

3.2 Objectives:

- Store user information
- Store product details
- Allow users to browse products
- Enable adding items to cart
- Allow removal of items from cart
- Check product availability before purchase
- Generate order records
- Update stock after purchase
- Store transaction details
- Allow admin to manage products
- Maintain data integrity
- Allow users to view previous orders

CHAPTER 3
DATABASE DESIGN

3.1 **ER Diagram**:



3.2 **Relational Schema**:

- USER(user_id PK, name, email, phone, address, password)
- PRODUCT(product_id PK, name, description, price, stock_qty, category)
- SHOPPING_CART(cart_id PK, customer_id FK, created_at, status)
- CART_ITEM(cart_item_id PK, cart_id FK, product_id FK, quantity)
- ORDER(order_id PK, customer_id FK, order_date, total_amount, status, payment_method)
- ORDER_ITEM(order_item_id PK, order_id FK, product_id FK, quantity, unit_price)
- CATEGORY(category_id, category_name)

3.3 Normalization:

- The database is normalized to avoid duplication and ensure data integrity.
- It follows three normal forms: **1NF, 2NF, and 3NF**.
- Ensures that all attributes depend only on the primary key and removes transitive dependencies.

First Normal Form (1NF)

Rules:

- No repeating groups
- Only atomic (single-valued) attributes

Implemented:

- All attributes in tables are atomic
- No multi-valued attributes are present
- Example: Products are stored separately instead of inside Order_Item

Second Normal Form (2NF)

Rules:

- Table must be in 1NF
- No partial dependency on composite primary keys

Implemented:

- Use of surrogate primary keys (e.g., *cart_item_id*, *order_item_id*)
- All non-key attributes fully depend on the primary key

Example:

- Order_Item(order_item_id, order_id, product_id, quantity, price)
- No partial dependency exists

Third Normal Form (3NF)

Rules:

- Table must be in 2NF
- No transitive dependency

Implemented:

- Data is split into separate tables to avoid redundancy
- Product details do not include category name directly

Example:

Incorrect:

    - Product(product_id, name, category_name)
- Correct:
    - Product(product_id, name, category_id)

- Category(category_id, category_name)

CHAPTER 4

DDL COMMANDS & SQL

```
CREATE TABLE USER (
   user_id INT PRIMARY KEY,
   name VARCHAR(50),
   email VARCHAR(50) UNIQUE,
   password VARCHAR(50),
   role VARCHAR(20)
);

CREATE TABLE CATEGORY (
   category_id INT PRIMARY KEY,
   category_name VARCHAR(50)
);

CREATE TABLE PRODUCT (
   product_id INT PRIMARY KEY,
   name VARCHAR(50),
   description VARCHAR(100),
   price DECIMAL(10,2),
   stock_quantity INT,
   category_id INT,
   FOREIGN KEY (category_id) REFERENCES CATEGORY(category_id)
);

CREATE TABLE CART (
   cart_id INT PRIMARY KEY,
```

```
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES USER(user_id)
);

CREATE TABLE CART_ITEM (
    cart_item_id INT PRIMARY KEY,
    cart_id INT,
    product_id INT,
    quantity INT,
    FOREIGN KEY (cart_id) REFERENCES CART(cart_id),
    FOREIGN KEY (product_id) REFERENCES PRODUCT(product_id)
);

CREATE TABLE ORDERS (
    order_id INT PRIMARY KEY,
    user_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (user_id) REFERENCES USER(user_id)
);

CREATE TABLE ORDER_ITEM (
    order_item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES ORDERS(order_id),
    FOREIGN KEY (product_id) REFERENCES PRODUCT(product_id)
);
```

SAMPLE INPUTS :

```
INSERT INTO users (name, email, password, role) VALUES ('Neha','neha@gmail.com','1234','customer');
INSERT INTO users (name, email, password, role) VALUES ('Akshita','akshita@gmail.com','1234','customer');
INSERT INTO users (name, email, password, role) VALUES ('Admin1','admin1@gmail.com','admin','admin');

INSERT INTO category (category_name) VALUES ('Electronics');
INSERT INTO category (category_name) VALUES ('Clothing');

INSERT INTO product (name, description, price, stock_quantity, category_id) VALUES ('Laptop', 'Dell Inspiron',60000,10,1) ;
INSERT INTO product (name, description, price, stock_quantity, category_id) VALUES ('Mobile', 'Samsung Galaxy',20000,15,1) ;

INSERT INTO cart (user_id) VALUES (1) ;
INSERT INTO cart (user_id) VALUES (2) ;

INSERT INTO cart_item (cart_id, product_id, quantity) VALUES (1,1,1) ;
INSERT INTO cart_item (cart_id, product_id, quantity) VALUES (1,3,2) ;

INSERT INTO orders (user_id, order_date, total_amount) VALUES (1, TO_DATE('2026-03-01','YYYY-MM-DD'), 63000) ;
INSERT INTO orders (user_id, order_date, total_amount) VALUES (2, TO_DATE('2026-03-05','YYYY-MM-DD'), 21500) ;

INSERT INTO order_item (order_id, product_id, quantity, price) VALUES (1,1,1,60000) ;
INSERT INTO order_item (order_id, product_id, quantity, price) VALUES (1,3,2,1500) ;
```

TABLES:

Figure: 4.1

```
SQL> desc users;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 USER_ID                                   NOT NULL NUMBER(38)
 NAME                                               VARCHAR2(50)
 EMAIL                                              VARCHAR2(50)
 PASSWORD                                           VARCHAR2(50)
 ROLE                                               VARCHAR2(20)

SQL> desc product;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PRODUCT_ID                                NOT NULL NUMBER(38)
 NAME                                               VARCHAR2(50)
 DESCRIPTION                                        VARCHAR2(100)
 PRICE                                              NUMBER(10,2)
 STOCK_QUANTITY                                     NUMBER(38)
 CATEGORY_ID                                        NUMBER(38)

SQL> desc cart;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CART_ID                                   NOT NULL NUMBER(38)
 USER_ID                                            NUMBER(38)

SQL> desc cart_item;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CART_ITEM_ID                              NOT NULL NUMBER(38)
 CART_ID                                            NUMBER(38)
 PRODUCT_ID                                         NUMBER(38)
 QUANTITY                                           NUMBER(38)

SQL> desc orders;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ORDER_ID                                  NOT NULL NUMBER(38)
 USER_ID                                            NUMBER(38)
 ORDER_DATE                                         DATE
 TOTAL_AMOUNT                                       NUMBER(10,2)

SQL> desc order_item;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ORDER_ITEM_ID                             NOT NULL NUMBER(38)
 ORDER_ID                                           NUMBER(38)
 PRODUCT_ID                                         NUMBER(38)
 QUANTITY                                           NUMBER(38)
 PRICE                                              NUMBER(10,2)
```

PROCEDURES/TRIGGERS IMPLEMENTED:

ADD_PRODUCT: procedure used to insert a new product into the **PRODUCT** table by the admin

```
CREATE OR REPLACE PROCEDURE add_product(
    pid NUMBER,
    pname VARCHAR2,
    pdesc VARCHAR2,
    pprice NUMBER,
    pstock NUMBER,
    cid NUMBER
) IS
BEGIN
    INSERT INTO PRODUCT VALUES(pid, pname, pdesc, pprice, pstock, cid);
END;
/
```

CART_ITEM_SEQ + CART_ITEM_TRIGGER: used to automatically generate unique IDs for cart items.

```
CREATE SEQUENCE cart_item_seq START WITH 1;

CREATE OR REPLACE TRIGGER cart_item_trigger
BEFORE INSERT ON CART_ITEM
FOR EACH ROW
BEGIN
    SELECT cart_item_seq.NEXTVAL INTO :NEW.cart_item_id FROM dual;
END;
/
```

CART_SEQ + CART_TRIGGER: used to automatically generate unique cart_id for the **CART** table.

```
CREATE SEQUENCE cart_seq START WITH 1;

CREATE OR REPLACE TRIGGER cart_trigger
BEFORE INSERT ON CART
FOR EACH ROW
BEGIN
    SELECT cart_seq.NEXTVAL INTO :NEW.cart_id FROM dual;
END;
/
```

CATEGORY_SEQ+ CATEGORY_TRIGGER: used to automatically generate unique IDs for the **CATEGORY** table.

```
CREATE SEQUENCE category_seq START WITH 1;

CREATE OR REPLACE TRIGGER category_trigger
BEFORE INSERT ON CATEGORY
FOR EACH ROW
BEGIN
    SELECT category_seq.NEXTVAL INTO :NEW.category_id FROM dual;
END;
/
```

ORDER_ITEM_SEQ+ ORDER_ITEM_TRIGGER: used to automatically generate unique IDs for the **ORDER_ITEM** table**.**

```
CREATE SEQUENCE order_item_seq START WITH 1;

CREATE OR REPLACE TRIGGER order_item_trigger
BEFORE INSERT ON ORDER_ITEM
FOR EACH ROW
BEGIN
    SELECT order_item_seq.NEXTVAL INTO :NEW.order_item_id FROM dual;
END;
/
```

ORDERS_SEQ+ ORDERS_TRIGGER: used to automatically generate unique IDs for the **ORDERS** table.

```
CREATE SEQUENCE orders_seq START WITH 1;

CREATE OR REPLACE TRIGGER orders_trigger
BEFORE INSERT ON ORDERS
FOR EACH ROW
BEGIN
    SELECT orders_seq.NEXTVAL INTO :NEW.order_id FROM dual;
END;
/
```

PRODUCT_SEQ + PRODUCT_TRIGGER: sequence and a trigger used to automatically generate unique IDs for the **PRODUCT** table.

```
CREATE SEQUENCE product_seq START WITH 1;

CREATE OR REPLACE TRIGGER product_trigger
BEFORE INSERT ON PRODUCT
FOR EACH ROW
BEGIN
    SELECT product_seq.NEXTVAL INTO :NEW.product_id FROM dual;
END;
/
```

DELETE_PRODUCT: procedure used to remove a product from the **PRODUCT** table by the admin.

```
CREATE OR REPLACE PROCEDURE delete_product(pid NUMBER) IS
BEGIN
    DELETE FROM PRODUCT WHERE product_id = pid;
END;
/
```

CHECK_STOCK: trigger used to validate product availability before placing an order.

```
CREATE OR REPLACE TRIGGER check_stock
BEFORE INSERT ON ORDER_ITEM
FOR EACH ROW
DECLARE
    available NUMBER;
BEGIN
    SELECT stock_quantity INTO available
    FROM PRODUCT
    WHERE product_id = :NEW.product_id;

    IF available < :NEW.quantity THEN
        RAISE_APPLICATION_ERROR(-20001, 'Not enough stock');
    END IF;
END;
/
```

UPDATE_PRODUCT: procedure used to modify the price and stock of an existing product in the **PRODUCT** table.

```
CREATE OR REPLACE PROCEDURE update_product(
    pid NUMBER,
    pprice NUMBER,
    pstock NUMBER
) IS
BEGIN
    UPDATE PRODUCT
    SET price = pprice,
        stock_quantity = pstock
    WHERE product_id = pid;
END;
/
```

UPDATE_STOCK: trigger used to automatically reduce product stock after an order is placed.

```
CREATE OR REPLACE TRIGGER update_stock
AFTER INSERT ON order_item
FOR EACH ROW
BEGIN
    UPDATE product
    SET stock_quantity = stock_quantity - :NEW.quantity
    WHERE product_id = :NEW.product_id;
END;
/
```

PLACE_ORDER: procedure used to convert a user's cart into a complete order.

```
CREATE OR REPLACE PROCEDURE place_order(p_uid IN NUMBER)
IS
    CURSOR cart_cursor IS
        SELECT ci.product_id, ci.quantity
        FROM cart_item ci
        JOIN cart c ON ci.cart_id = c.cart_id
        WHERE c.user_id = p_uid;

    v_pid product.product_id%TYPE;
    v_qty NUMBER;
    v_price product.price%TYPE;
    v_total NUMBER := 0;
    v_oid orders.order_id%TYPE;

BEGIN
    INSERT INTO orders (user_id, order_date, total_amount)
    VALUES (p_uid, SYSDATE, 0)
    RETURNING order_id INTO v_oid;

    FOR rec IN cart_cursor LOOP
        v_pid := rec.product_id;
        v_qty := rec.quantity;

        BEGIN
            SELECT price INTO v_price
            FROM product
            WHERE product_id = v_pid;

        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                DBMS_OUTPUT.PUT_LINE('Product not found: ' || v_pid);
                CONTINUE;
        END;

        INSERT INTO order_item(order_id, product_id, quantity, price)
        VALUES (v_oid, v_pid, v_qty, v_price);

        UPDATE product
        SET stock_quantity = stock_quantity - v_qty
        WHERE product_id = v_pid;

        v_total := v_total + (v_price * v_qty);
    END LOOP;
```

```
    UPDATE orders
    SET total_amount = v_total
    WHERE order_id = v_oid;

    DELETE FROM cart_item
    WHERE cart_id IN (
        SELECT cart_id FROM cart WHERE user_id = p_uid
    );

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

ADD_TO_CART: procedure used to add an item to the cart**.**

```
CREATE OR REPLACE PROCEDURE add_to_cart(
    p_cart_id NUMBER,
    p_pid NUMBER,
    p_qty NUMBER
)
IS
BEGIN
    INSERT INTO cart_item(cart_id, product_id, quantity)
    VALUES (p_cart_id, p_pid, p_qty);
END;
/
```

REMOVE_FROM_CART: procedure used to remove an item from the cart.

```
CREATE OR REPLACE PROCEDURE remove_from_cart(
    p_cart_id NUMBER,
    p_pid NUMBER
)
IS
BEGIN
    DELETE FROM cart_item
    WHERE cart_id = p_cart_id AND product_id = p_pid;
END;
/
```

SEARCH_PRODUCT : procedure used to search a product in the PRODUCT table.

```
set serveroutput on;
CREATE OR REPLACE PROCEDURE search_product(p_name VARCHAR2)
IS
BEGIN
    FOR rec IN (
        SELECT * FROM product
        WHERE LOWER(name) LIKE LOWER('%' || p_name || '%')
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(rec.name || ' - ' || rec.price);
    END LOOP;
END;
/
```

VIEW_ORDERS : procedure used to view past orders of that customer.

```
set serveroutput on;
CREATE OR REPLACE PROCEDURE view_orders(p_uid NUMBER)
IS
BEGIN
    FOR rec IN (
        SELECT * FROM orders
        WHERE user_id = p_uid
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Order ID: ' || rec.order_id ||
                             ' Total: ' || rec.total_amount);
    END LOOP;
END;
/
```

CHAPTER 5
RESULTS & SNAPSHOTS

Screenshots of table creation and query execution are included.

LOGIN PAGE:

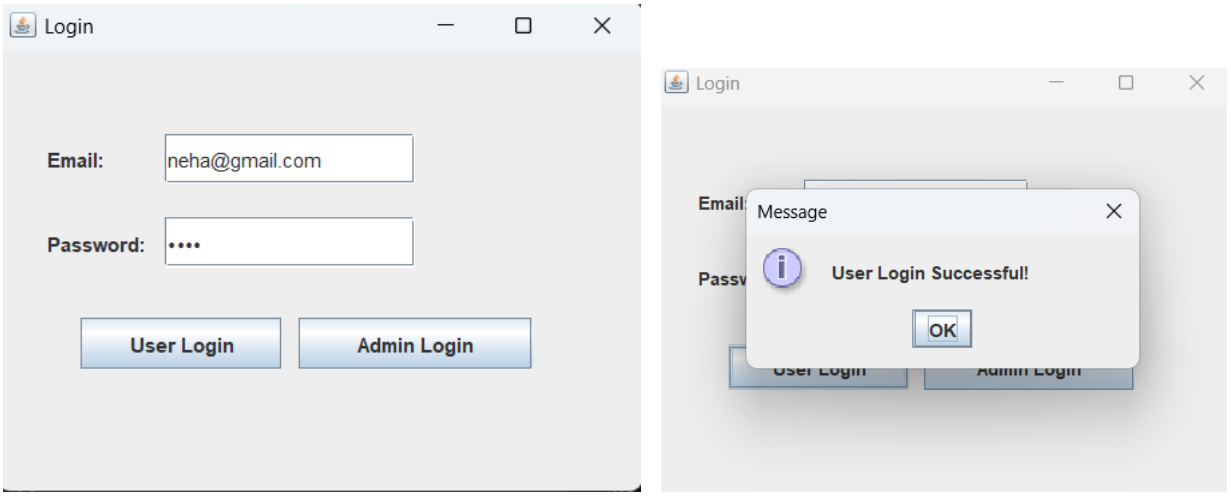


Figure 5.1

PRODUCT PAGE:

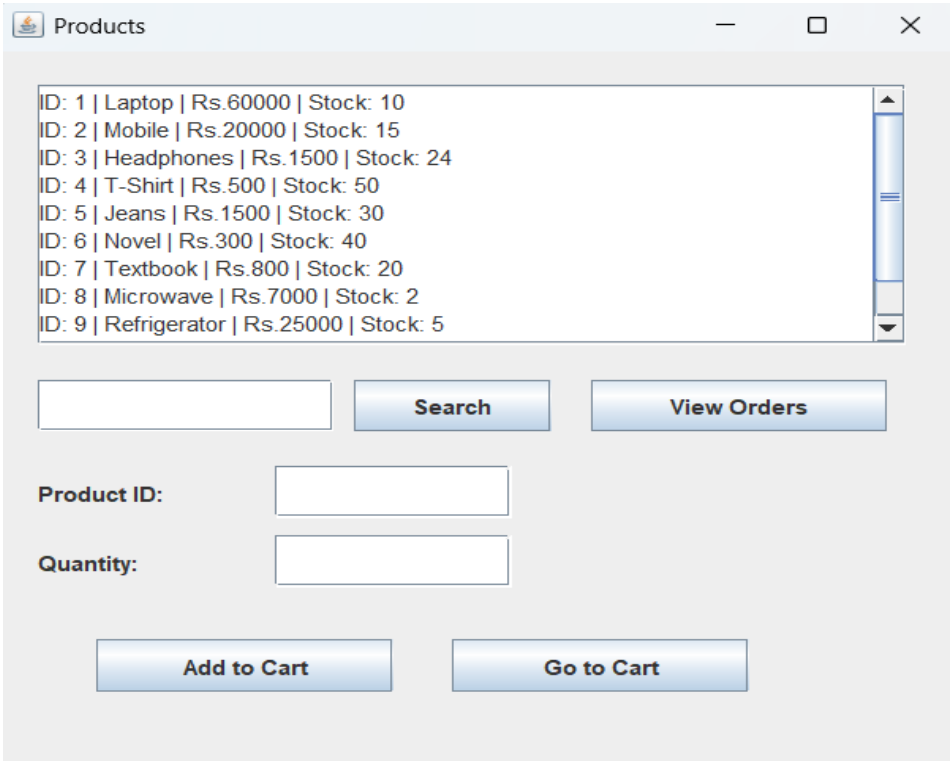


Figure 5.2

SEARCHING A PRODUCT:

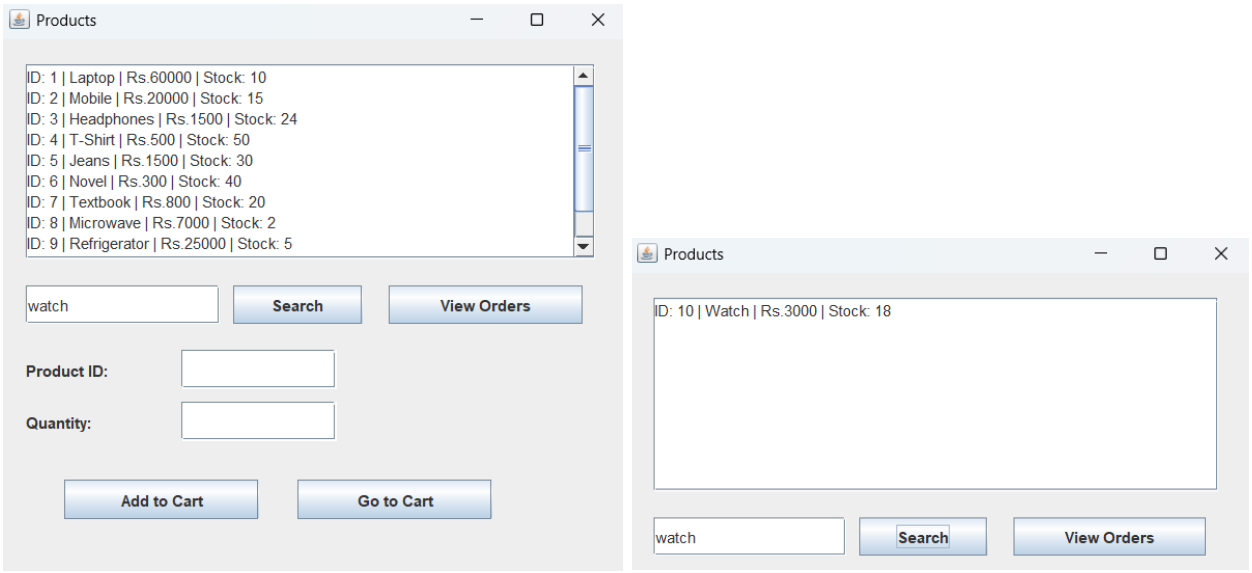


Figure 5.3

ADDING A PRODUCT INTO THE CART:

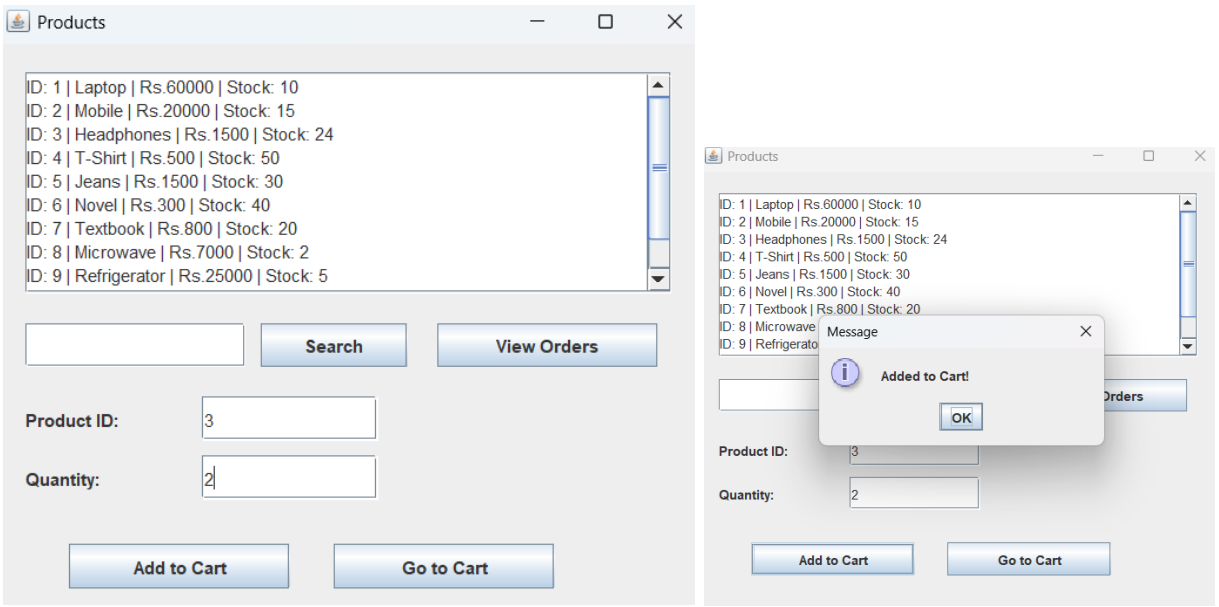


Figure 5.4

SHOPPING CART:

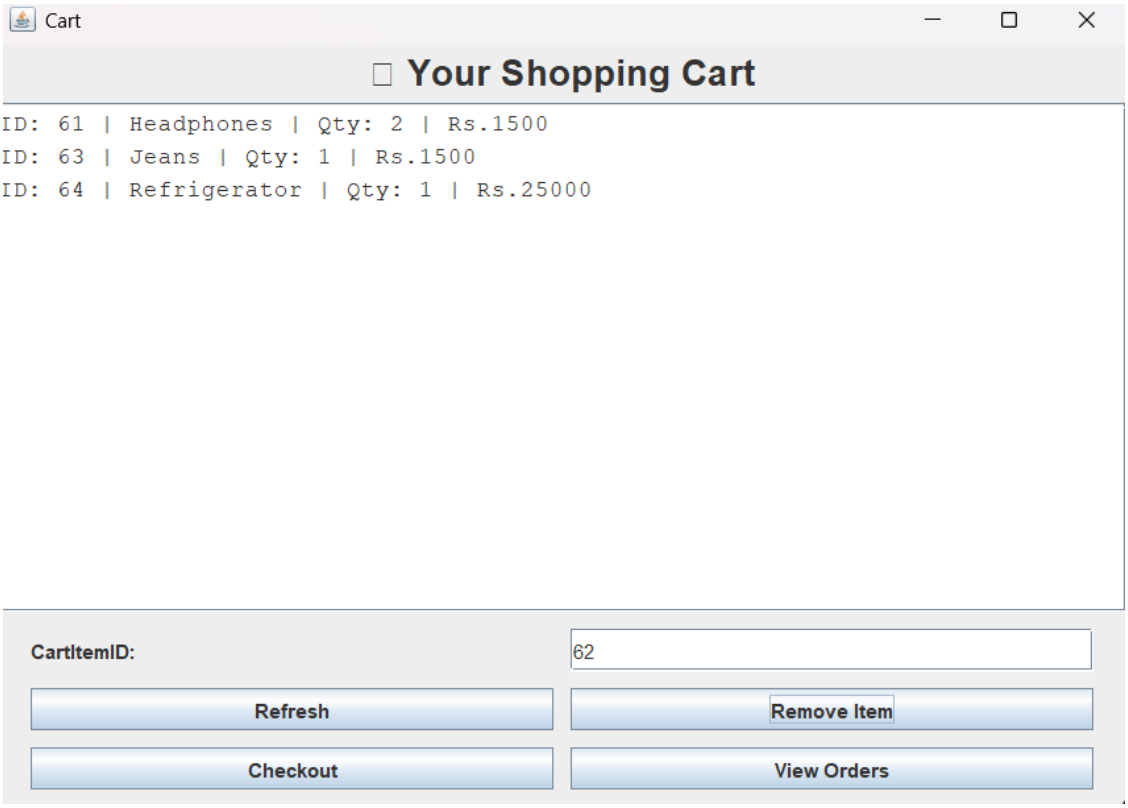


Figure 5.5

REMOVING AN ITEM:

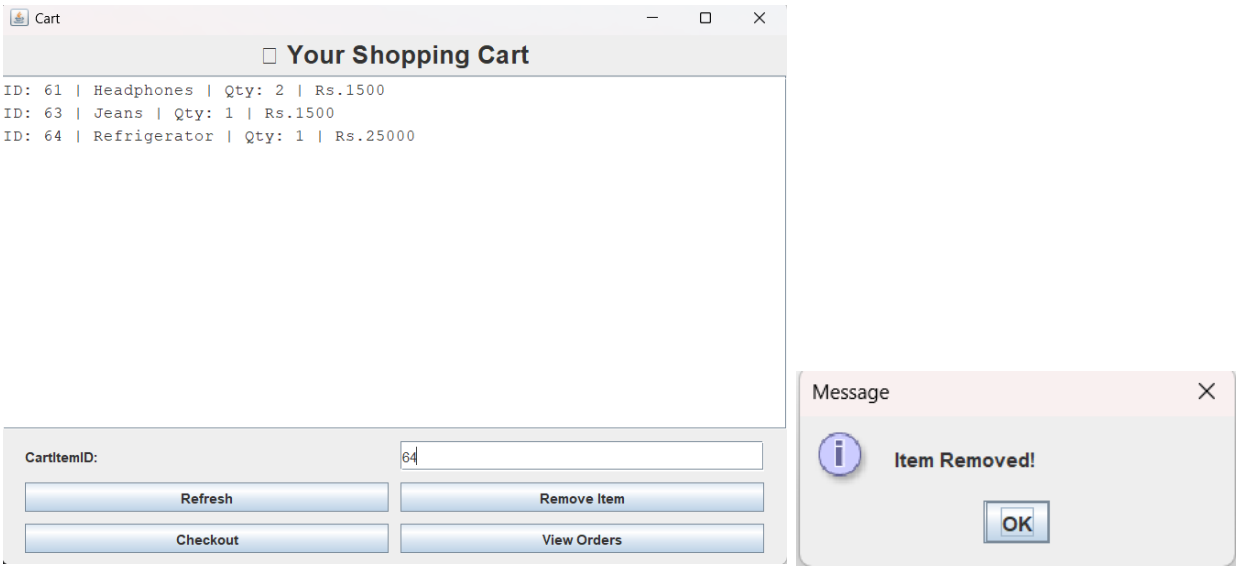
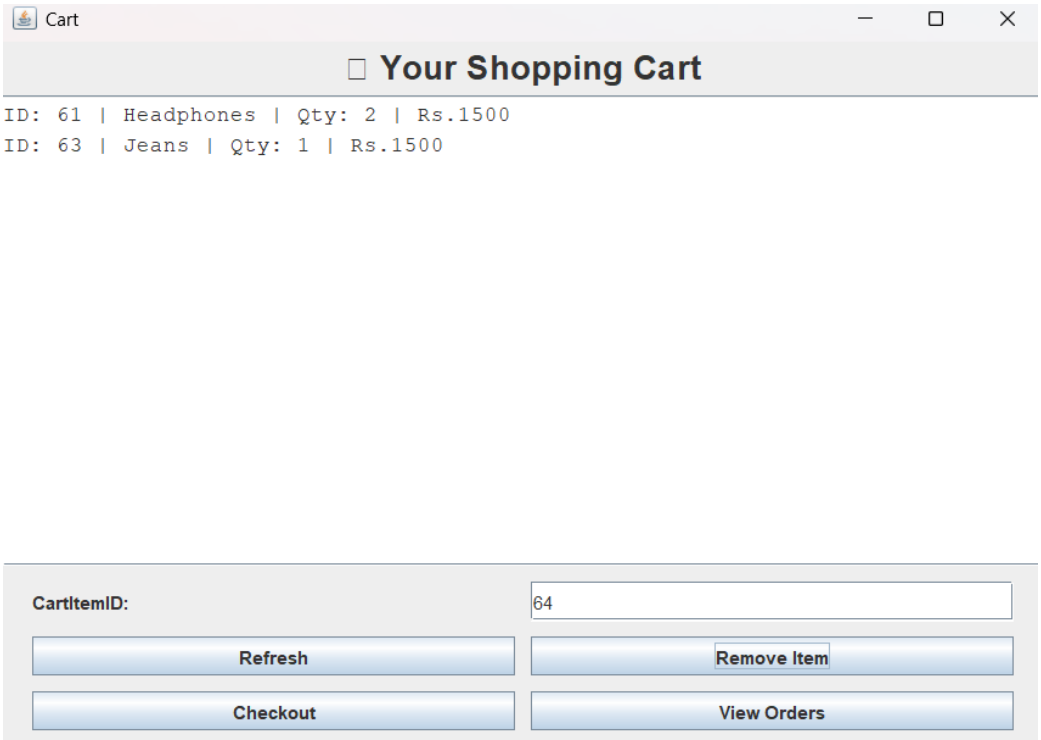


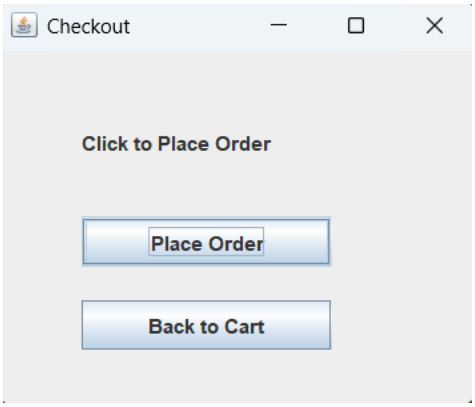
Figure 5.6




```
SQL> select * from cart_item;

CART_ITEM_ID    CART_ID PRODUCT_ID   QUANTITY
------------ ---------- ---------- ----------
          61          1          3          2
          63          1          5          1
```

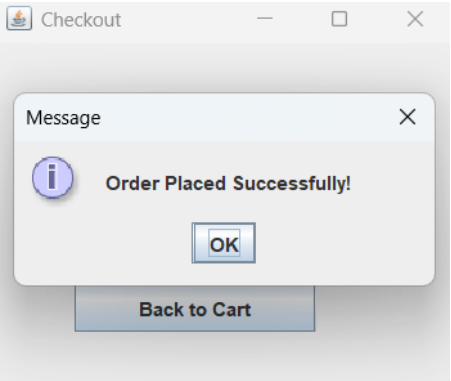
CHECKOUT PAGE:




```
SQL> select * from order_item;

no rows selected
```

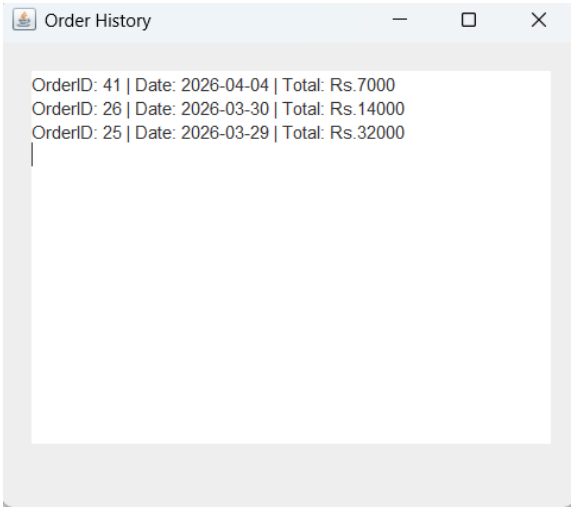
Figure 5.7




```
SQL> select * from order_item;

ORDER_ITEM_ID   ORDER_ID PRODUCT_ID   QUANTITY      PRICE
------------- ---------- ---------- ---------- ----------
           61         61          3          2       1500
           62         61          5          1       1500
```

ORDER HISTORY PAGE:

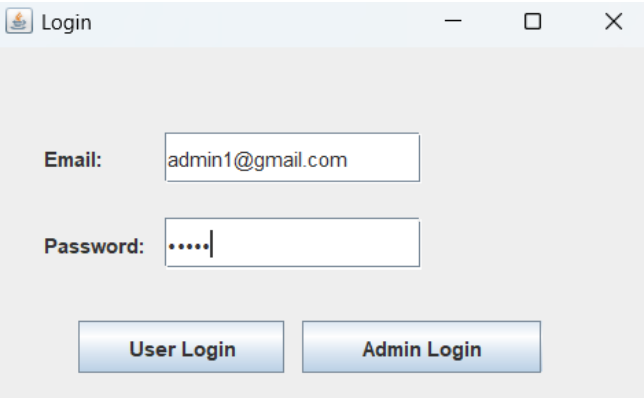


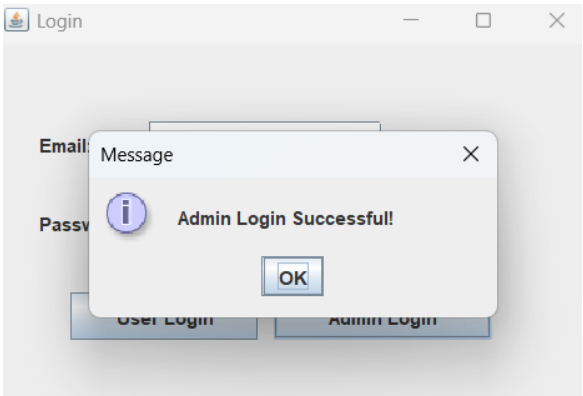

```
SQL> select * from orders;

  ORDER_ID    USER_ID ORDER_DAT TOTAL_AMOUNT
---------- ---------- --------- ------------
        41          1 04-APR-26         7000
        26          1 30-MAR-26        14000
        25          1 29-MAR-26        32000
```

Figure 5.8

ADMIN PAGE:





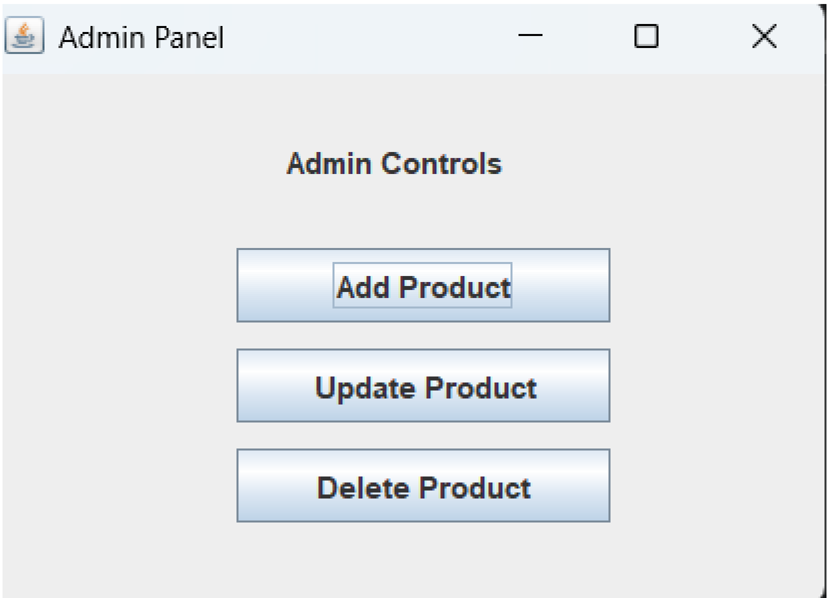


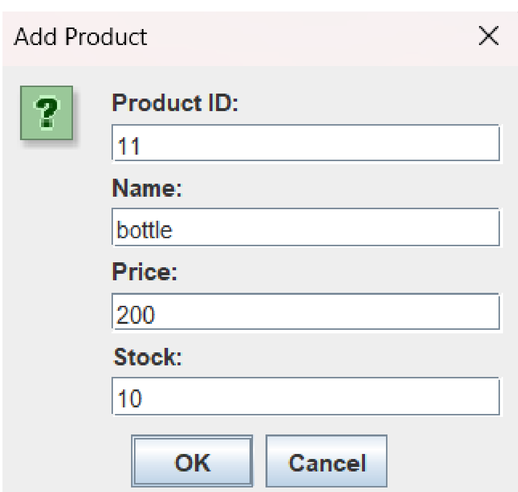
Figure 5.9

ADDING A PRODUCT:

```
SQL> select * from product;

PRODUCT_ID NAME                                               DESCRIPTION                                                   PRICE STOCK_QUANTITY CATEGORY_ID
---------- -------------------------------------------------- ----------------------------------------------------------- ----- -------
--- -------------- -----------
         1 Laptop                                             Dell Inspiron                                                 60000             10           1
         2 Mobile                                             Samsung Galaxy                                                20000             15           1
         3 Headphones                                         Boat Headset                                                   1500             20           5
         4 T-Shirt                                            Cotton T-Shirt                                                  500             50           2
         5 Jeans                                              Denim Jeans                                                    1500             28           2
         6 Novel                                              Fiction Book                                                    300             40           3
         7 Textbook                                           DBMS Book                                                       800             20           3
         8 Microwave                                          LG Microwave                                                   7000              2           4
         9 Refrigerator                                       Samsung Fridge                                                25000              5           4
        10 Watch                                              Smart Watch                                                    3000             18           5

10 rows selected.
```



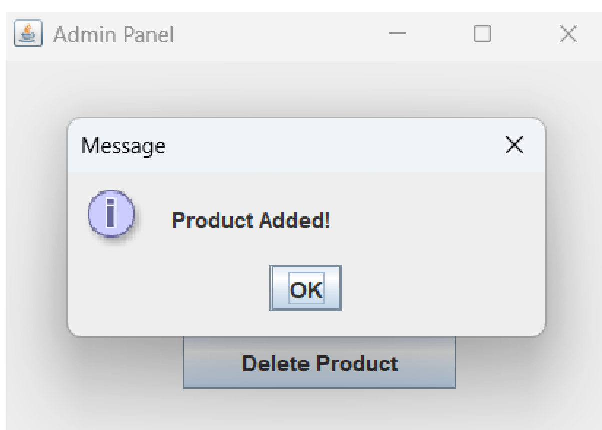


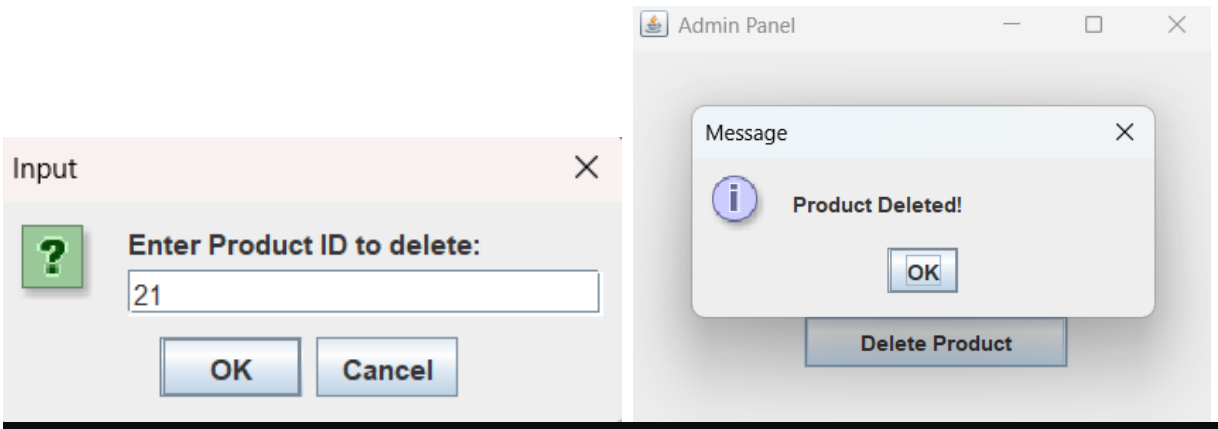
Figure 5.10

```
SQL> select * from product;

PRODUCT_ID NAME            DESCRIPTION              PRICE STOCK_QUANTITY CATEGORY_ID
---------- --------------- ------------------------ ------ -------------- -----------
        21 bottle          New Product                200             10           1
         1 Laptop          Dell Inspiron            60000             10           1
         2 Mobile          Samsung Galaxy           20000             15           1
         3 Headphones      Boat Headset              1500             20           5
         4 T-Shirt         Cotton T-Shirt             500             50           2
         5 Jeans           Denim Jeans               1500             28           2
         6 Novel           Fiction Book               300             40           3
         7 Textbook        DBMS Book                  800             20           3
         8 Microwave       LG Microwave              7000              2           4
         9 Refrigerator    Samsung Fridge           25000              5           4
        10 Watch           Smart Watch               3000             18           5

11 rows selected.
```

DELETING A PRODUCT:



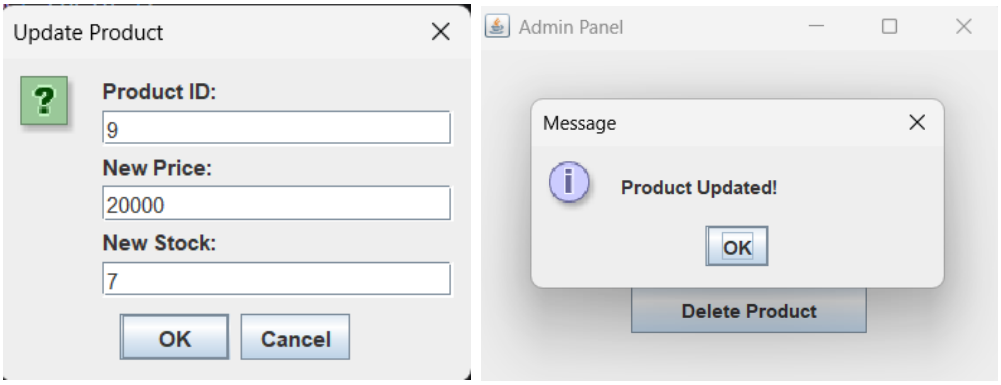

```
SQL> select * from product;

PRODUCT_ID NAME            DESCRIPTION              PRICE STOCK_QUANTITY CATEGORY_ID
---------- --------------- ------------------------ ------ -------------- -----------
         1 Laptop          Dell Inspiron            60000             10           1
         2 Mobile          Samsung Galaxy           20000             15           1
         3 Headphones      Boat Headset              1500             20           5
         4 T-Shirt         Cotton T-Shirt             500             50           2
         5 Jeans           Denim Jeans               1500             28           2
         6 Novel           Fiction Book               300             40           3
         7 Textbook        DBMS Book                  800             20           3
         8 Microwave       LG Microwave              7000              2           4
         9 Refrigerator    Samsung Fridge           25000              5           4
        10 Watch           Smart Watch               3000             18           5

10 rows selected.
```

Figure 5.11

UPDATING A PRODUCT:




```
PRODUCT_ID NAME           DESCRIPTION              PRICE STOCK_QUANTITY CATEGORY_ID
---------- -------------- ------------------------ ------ -------------- -----------
         9 Refrigerator   Samsung Fridge           20000              7           4
```

Figure 5.12

CHAPTER 6
CONCLUSION, LIMITATIONS & FUTURE WORK

6.1 Conclusion:

The Online Shopping Cart Management System shows how to design and build a database for managing online shopping activities. The system handles user information, product details, shopping cart tasks, and order processing in an organized way.

By using a relational database, the system keeps data consistent, cuts down on repetition, and makes data retrieval easy. Setting up relationships between entities like User, Product, Cart, and Order helps maintain transaction integrity.

The system also checks product availability before purchase and updates stock levels after an order is completed, which prevents invalid transactions. Overall, the project showcases how DBMS concepts can be applied in real life.

6.2 Limitations:

- There is no real-time-payment-gateway provision to process transactions.
- Encryption and authentication mechanisms are limited for security features.
- The user interface is rudimentary and lacks advanced user interaction.
- Lastly, it did not support real-time notifications and order tracking.
- The scalability is limited to handling many concurrent users.

6.3 Future Work:

- Integration of a secure online payment gateway for a full transaction process.

- Authentication and encryption techniques implementation to increase security.
- Development of a user-friendly graphical interface for enhanced usability.
- Inclusion of real-time features like order tracking and notifications.
- Refinement of the system for scalability for large-scale uses.
- Addition of sophisticated functionalities like product recommendations and user reviews.

CHAPTER 7
REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.

[3] Oracle Corporation, "Oracle Database SQL Language Reference," [Online]. Available: https://docs.oracle.com

[4] "SQL Tutorial," W3Schools, [Online]. Available: https://www.w3schools.com/sql/

[5] "Database Design Basics," Microsoft Docs, [Online]. Available: https://learn.microsoft.com

[6] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed., Pearson, 2015.